Researchsprint_code

2026-04-16

Code for AMC Research Sprint

Imports

```r
library(tidyverse)
```

```
## Warning: package 'ggplot2' was built under R version 4.5.2

## Warning: package 'tibble' was built under R version 4.5.2

## Warning: package 'tidyr' was built under R version 4.5.2

## Warning: package 'readr' was built under R version 4.5.2

## Warning: package 'purrr' was built under R version 4.5.2

## Warning: package 'dplyr' was built under R version 4.5.2

## Warning: package 'lubridate' was built under R version 4.5.2

## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.2.0     v readr     2.1.6
## v forcats   1.0.1     v stringr   1.6.0
## v ggplot2   4.0.2     v tibble    3.3.1
## v lubridate 1.9.5     v tidyr     1.3.2
## v purrr     1.2.1
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```r
library(stringr)
library(ggrepel)
```

```
## Warning: package 'ggrepel' was built under R version 4.5.2
```

```r
library(modelsummary)
```

```
## Warning: package 'modelsummary' was built under R version 4.5.2
```

```r
library(ggplot2)
library(scales)
```

```
##
## Attaching package: 'scales'
##
## The following object is masked from 'package:purrr':
##
##     discard
##
## The following object is masked from 'package:readr':
##
##     col_factor
```

Folder paths

```r
raw_data_path <- "../data"
cleaned_data_path <- "data_cleaned"
fig_path <- "figs"
```

Read in data

```r
vercom <- read_csv(
  file.path(raw_data_path, "amcdata_vercom_V2.csv"),
  na = c("","N/A"),
  show_col_types = FALSE
)
agreement_info <- read_csv(
  file.path(raw_data_path, "amcdata_agreement_info_V2.csv"),
  na = c("","N/A"),
  show_col_types = FALSE
)
weapons_bans <- read_csv(
  file.path(raw_data_path, "amcdata_weapons_facilities_V2.csv"),
  na = c("","N/A"),
  show_col_types = FALSE,
  name_repair = "minimal"
) |> select(!where(~ all(is.na(.)))) # remove trailing columns that are empty
```

Construct Verification Stringency Index

Now, the main challenge is to construct a verification stringency index. For this, we first engineer some features, which we will then combine into one weighted index.

Step 1: Feature engineering

1. Mechanism Duration We re-code the duration of a given mechanism from the *verified_compliance_mechanism_duratio* variables. We follow the order in the original data set, but modify it so that mechanisms with a set time of over 10 years, are regarded as more intense than mechanisms with an obligations timeline. This results in an ordinal variable in the following order (from least to most intense):

0. Set time (< 10 years)
1. Obligations timeline
2. Set time (>= 10 years)
3. Agreement duration

Due to time constraints, the "other" observations were coded as 0.

To construct this, we first need to compute the duration number in years:

```
vercom <- vercom |>
  mutate(
    # Extract numeric part
    duration_number = as.numeric(
      str_extract(verified_compliance_mechanism_duration_set, "[0-9]+")),
    duration_unit = str_extract(
      verified_compliance_mechanism_duration_set, "[a-z]+"),
    # Convert to years
    duration_years = case_when(
      duration_unit == "days"  ~ duration_number / 365,
      duration_unit == "year"  ~ duration_number,   # handles "1 year"
      duration_unit == "years" ~ duration_number,
      TRUE ~ NA_real_
    )
  )
```

Then, re-code the 1 "other" observation to be a 4 instead of a 3 (this is a mistake in the data set that we need to control for), and then subtract 1 for all observation to comply with the original codebook:

```
vercom <- vercom |>
  mutate(
    verified_compliance_mechanism_duration = case_when(
      !is.na(verified_compliance_mechanism_duration_other) ~ 4,
      TRUE ~ verified_compliance_mechanism_duration
    ) - 1
  )
```

Now we can compute the mechanism_duration variable, and remove the temporary variables:

```
vercom <- vercom |>
  mutate(
    mechanism_duration = case_when(
      # Set time (< 10 years)
      verified_compliance_mechanism_duration == 0 & duration_years < 10   ~ 0,
      # Obligations timeline
      verified_compliance_mechanism_duration == 1                         ~ 1,
      # Set time (>= 10 years)
      verified_compliance_mechanism_duration == 0 & duration_years >= 10  ~ 2,
      # Agreement duration
      verified_compliance_mechanism_duration == 2                         ~ 3,
      # Other
      verified_compliance_mechanism_duration == 3                         ~ 0,
      # Else case
      TRUE                                                                ~ 0
```

```
    )) |>
  select(
    -duration_number, -duration_unit, -duration_years
  )
```

2. Trigger mechanism For our trigger mechanism indicator we combine the features *verified_compliance_mechanism_trigger_type* and *verified_compliance_mechanism_agreement_trigger* into one, to arrive at the following ordinal variable:

1. State-party notification (*trigger_type*)
2. Agreement association (*trigger_type*), one time (*agreement_trigger*)
3. Circumstance (*trigger_type*)
4. Agreement (*trigger_type*), quota (*agreement_trigger*)
5. Agreement (*trigger_type*), cyclical (*agreement_trigger*)
6. Agreement association (*trigger_type*)
7. Agreement (*trigger_type*), continuous (*agreement_trigger*)
8. State-party request (*trigger_type*)

Due to time constraints, the "other" observations were coded as 1.

To construct this, we re-code the one "7" value for *verified_compliance_mechanism_trigger_type* to be a 5 (other) instead, and we re-code the one "4" value for *verified_compliance_mechanism_agreement_trigger* to be a 0 (one time) instead; (there are mistakes in the data set that we need to control for):

```
vercom <- vercom |>
  mutate(
    verified_compliance_mechanism_trigger_type = case_when(
      verified_compliance_mechanism_trigger_type == 7 ~ 5,
      TRUE ~ verified_compliance_mechanism_trigger_type
    ),
    verified_compliance_mechanism_agreement_trigger_type = case_when(
      verified_compliance_mechanism_agreement_trigger_type == 4 ~ 1,
      TRUE ~ verified_compliance_mechanism_agreement_trigger_type
    )
  )
```

Then, we can re-code the variable properly:

```
vercom <- vercom |>
  mutate(
    trigger_mechanism = case_when(
      # 1: state party notification trigger
      verified_compliance_mechanism_trigger_type == 1 ~ 1,
      # 2: agreement, one time
      verified_compliance_mechanism_trigger_type == 0 &
        verified_compliance_mechanism_agreement_trigger_type == 0 ~ 2,
      # 3: event trigger (circumstance)
      verified_compliance_mechanism_trigger_type == 3 ~ 3,
      # 4: agreement, quota
      verified_compliance_mechanism_trigger_type == 0 &
        verified_compliance_mechanism_agreement_trigger_type == 1 ~ 4,
      # 5: agreement, cyclical
```

```
      verified_compliance_mechanism_trigger_type == 0 &
        verified_compliance_mechanism_agreement_trigger_type == 2 ~ 5,
      # 6: agreement association trigger
      verified_compliance_mechanism_trigger_type == 4 ~ 6,
      # 7: agreement, continuous
      verified_compliance_mechanism_trigger_type == 0 &
        verified_compliance_mechanism_agreement_trigger_type == 3 ~ 7,
      # 8: state request
      verified_compliance_mechanism_trigger_type == 2 ~ 8,
      # other; becomes 1 (this is a rough recoding, should be specified better)
      TRUE ~ 1
    ))
```

3. Access level We re-code the access level as an ordinal variable based on the individual levels of access, using the variables *verified_compliance_mechanism_area_access, verified_compliance_mechanism_facility_access, verified_compliance_mechanism_item_access* & *verified_compliance_mechanism_item_section_access.* We conceive of access_intensity in the following order:

0. No access
1. Item section access
2. Item access
3. Facility access
4. Area access

Construction:

```
vercom <- vercom |>
  mutate(
    access_level = case_when(
      # 4: area access
      verified_compliance_mechanism_area_access == 1 ~ 4,
      # 3: facility access
      verified_compliance_mechanism_facility_access == 1 ~ 3,
      # 2: item access
      verified_compliance_mechanism_item_access == 1 ~ 2,
      # 1: item section acess
      verified_compliance_mechanism_item_section_access == 1 ~ 1,
      # 0: no access
      TRUE ~ 0
    )
  )
```

4. Phase access level We re-code the phase access level from a group of individual variables regarding access in a specific phase, using *verified_compliance_mechanism_possession, verified_compliance_mechanism_testing, verified_compliance_mechanism_production, verified_compliance_mechanism_use & verified_compliance_mechanism_general.* These variables are all coded as binary in the data set, but we combine them into a single variable using a weighted combination. The phases possession, use and general are given weight 2, and testing and production are given weight 1, in terms of verification intensity.

First, we replace the access variables that are NAs with 0:

```r
access_variables <- c(
  "verified_compliance_mechanism_possession",
  "verified_compliance_mechanism_testing",
  "verified_compliance_mechanism_production",
  "verified_compliance_mechanism_use",
  "verified_compliance_mechanism_general"
)

vercom <- vercom |>
  mutate(across(all_of(access_variables), ~ replace_na(.x, 0)))
```

And then we compute the weighted combination of these different levels of access:

```r
vercom <- vercom |>
  mutate(
    phase_access_level = ((
      verified_compliance_mechanism_possession * 2 + # possession weight = 2
      verified_compliance_mechanism_testing * 1 + # testing weight = 1
      verified_compliance_mechanism_production * 1 + # production weight = 1
      verified_compliance_mechanism_use * 2 + # use weight = 2
      verified_compliance_mechanism_general * 2 # general weight = 2
    ) / 8) # divide by total weigth (2+1+1+2+2=8)
  )
```

5. Verified Compliance Mechanism The verified compliance mechanism was copied as present in the dataset from the *verified_compliance_mechanism_category* variable.

6. Inspector Type We construct an ordinal inspector type variable by slightly re-ordering the *verified_compliance_mechanism_inspector_type* variable, so it becomes:

0. Utilized association
1. Agreement-established association
2. Party on party
3. Other

Due to time constrains, "other" was coded as 3.

To construct:

```r
vercom <- vercom |>
  mutate(
    inspector_type = case_when(
      # 0: Utilized association
      verified_compliance_mechanism_inspector_type == 2 ~ 0,
      # 1: Agreement-established association
      verified_compliance_mechanism_inspector_type == 1 ~ 1,
      # 2: party on party
      verified_compliance_mechanism_inspector_type == 0 ~ 2,
      # 3: other; becomes 3 (this is a rough recoding, should be specified better)
      verified_compliance_mechanism_inspector_type == 3 ~ 3,
    )
  )
```

Step 2: Scale Features

We then scale all 6 ordinal variables from 0 to 1:

```
scale_vars <- c(
  "verified_compliance_mechanism_category",
  "access_level",
  "mechanism_duration",
  "trigger_mechanism",
  "phase_access_level",
  "inspector_type"
)

vercom <- vercom |>
  mutate(
    across(all_of(scale_vars), ~ (.x - min(.x, na.rm = TRUE)) /
                                  (max(.x, na.rm = TRUE) - min(.x, na.rm = TRUE)))
  )
```

Step 3: Weighting Features

And then we weigh the features to create a combined index, using the following weighting:

Most important (25%)

1. verified_compliance_mechanism_category
2. access_level

Slightly less important (12.5%)

3. Trigger mechanism
4. Mechanism duration
5. Phase access
6. Inspector type

```
w <- c(
  verified_compliance_mechanism_category = 0.25,
  access_level = 0.25,
  mechanism_duration = 0.125,
  trigger_mechanism = 0.125,
  phase_access_level  = 0.125,
  inspector_type = 0.125
)
```

And apply:

```
vercom <- vercom |>
  mutate(
    verification_intensity =
       verified_compliance_mechanism_category * w["verified_compliance_mechanism_category"]  +
       access_level    * w["access_level"]    +
       mechanism_duration * w["mechanism_duration"] +
```

```
      trigger_mechanism   * w["trigger_mechanism"]   +
      phase_access_level * w["phase_access_level"] +
      inspector_type * w["inspector_type"]
  )
```

Step 4: Merge with Agreements data

Aggregate the verification data from the mechanism level to the agreement level, in different ways (keeping the mean, max and sum of verification intensity for each agreement):

```
grouped_verification_scores <- vercom |>
  group_by(agreement_id) |>
  summarise(
    mean_verification_intensity = mean(verification_intensity, na.rm = TRUE),
    max_verification_intensity  = max(verification_intensity,  na.rm = TRUE),
    sum_verification_intensity  = sum(verification_intensity,  na.rm = TRUE),
    n_verification_mechanisms = n(),
    .groups = "drop"
  )
```

And then merge with the agreement data:

```
merged_data <- agreement_info |>
  left_join(grouped_verification_scores, by = "agreement_id") |>
  mutate(
    across(c(
      mean_verification_intensity, max_verification_intensity, sum_verification_intensity
    ), ~ replace_na(.x, 0)),
    n_verification_mechanisms = replace_na(n_verification_mechanisms, 0)
  )
```

Step 5: Extend Measure With Consultation and Demonstration

We also make an extended index that takes into account consultations and demonstrations as well, defined as follows:

- no consultation -> 0
- nr(consultation_mechanism) > *1 –> 1*
- nr(demonstrated_mechanism) > *1 -> 2*
- nr(verification_mechanisms) > 1 –> 2*max_verification_intensity

```
merged_data <- merged_data |>
  mutate(
    # fill in NAs as 0
    demonstrated_compliance_mechanism_nr = replace_na(demonstrated_compliance_mechanism_nr, 0),
    consultation_mechanism = replace_na(consultation_mechanism, 0)
  ) |>
  mutate(
    extended_verification_intensity = case_when(
```

```
      # if there is any verification, scale verification intensity
      # between 2 and 4
      n_verification_mechanisms > 0 ~ 2 + max_verification_intensity * 2,
      demonstrated_compliance_mechanism_nr > 0 ~ 2,
      consultation_mechanism == 1 ~ 1,
      TRUE ~ 0
    ))
```

Step 6: Compute Participation Measures

For participation measures, we use:

- *nr_states_parties_total*
- *nr_states_parties_ratification*
- *ratification_percentage* = (*nr_states_parties_ratification* / *nr_states_parties_total*)

First we need to fill in some gaps in the data

```
merged_data <- merged_data |>
  mutate(
    nr_states_parties_ratification = replace_na(nr_states_parties_ratification, 0),
    nr_states_parties_total = case_when(
      !is.na(nr_states_parties_total) ~ nr_states_parties_total,
      !is.na(nr_signatory_states)     ~ nr_signatory_states,
      TRUE                            ~ 0
    )
  )
```

And then we can compute the ratification percentage:

```
 merged_data <- merged_data |>
   mutate(
     ratification_percentage = nr_states_parties_ratification / nr_states_parties_total
   )
```

Step 7: Compute Ban Intensity Variable

To control for scope (approximately), we construct a variable *ban_intensity*, which we define as the number of categories in which a certain weapon is banned, summed for each agreement over all banned weapons. For this we use the following ban variables from the *weapons_bans* dataset:

```
ban_variables = c(
  "ban_development",
  "ban_testing",
  "ban_production",
  "ban_acquisition",
  "ban_possession",
  "ban_possession_deployed",
  "ban_transfer",
```

```
  "ban_use",
  "ban_disposal"
)
```

And then sum the number of categories in which an items is banned for each agreement-weapon unit:

```
weapons_bans <- weapons_bans |>
  mutate(
      ban_scope = rowSums(across(all_of(ban_variables)), na.rm = TRUE),
      ban_scope = ban_scope / length(ban_variables)
  )
```

Finally, aggregate (sum) the ban_scope for each agreement into a ban_intensity measure:

```
aggregated_bans <- weapons_bans |>
  group_by(agreement_id) |>
  summarise(
    ban_intensity = sum(ban_scope),
    .groups = "drop"
  )
```

And we then join this variable back onto the original data:

```
merged_data <- merged_data |>
  left_join(aggregated_bans, by = "agreement_id") |>
  mutate(
    ban_intensity = replace_na(ban_intensity, 0)
  )
```

Step 8: Save cleaned data

Then, save the new merged data set as a csv for later use and replicability purposes:

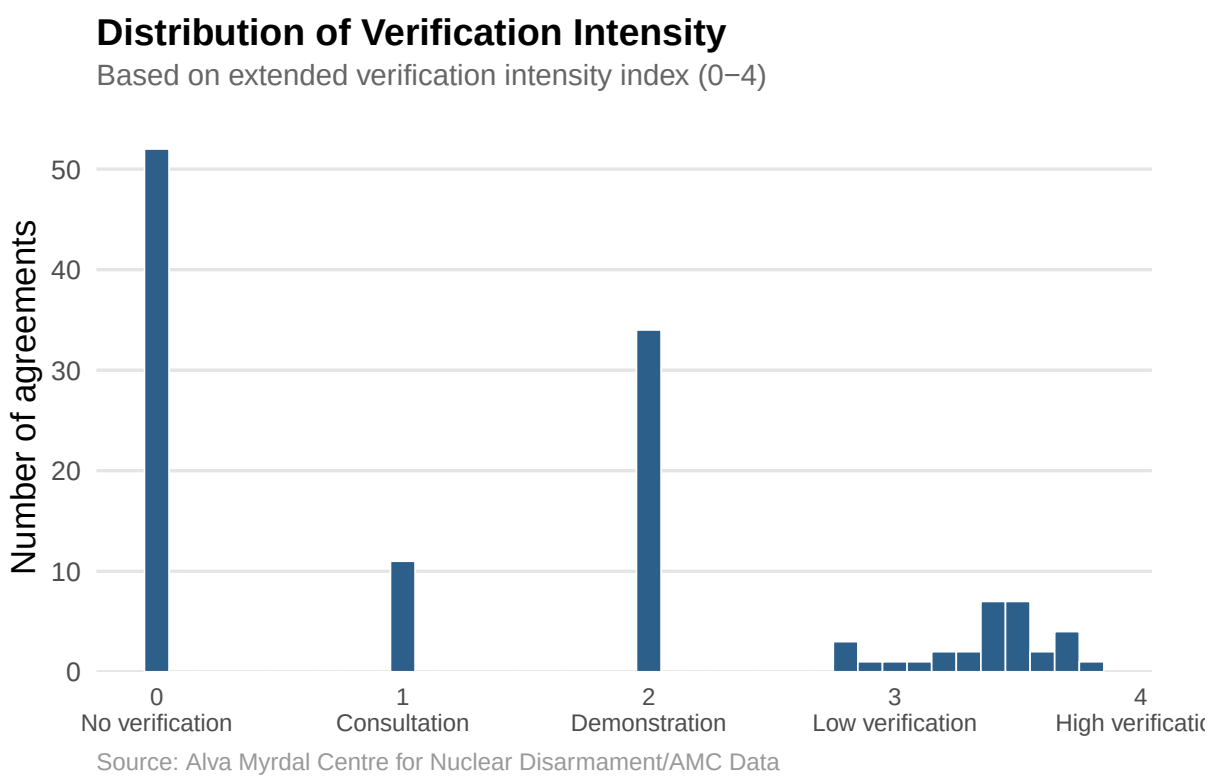
```
write.csv(merged_data, file.path(cleaned_data_path, "merged_data.csv"))
```

Explore Verification Intensity Variable

We can now create some visualizations, such as the distribution of verification intensity:

```
merged_data |>
  ggplot(aes(x = extended_verification_intensity)) +
  geom_histogram(
    binwidth = 0.1,
    fill     = "#2C5F8A",
    color    = "white",
    linewidth = 0.3
  ) +
  scale_x_continuous(
    breaks = 0:4,
    labels = c("0\nNo verification", "1\nConsultation", "2\nDemonstration",
```

```r
              "3\nLow verification", "4\nHigh verification")
  ) +
  scale_y_continuous(expand = expansion(mult = c(0, 0.05))) +
  labs(
    title    = "Distribution of Verification Intensity",
    subtitle = "Based on extended verification intensity index (0-4)",
    caption  = "Source: Alva Myrdal Centre for Nuclear Disarmament/AMC Data",
    x        = NULL,
    y        = "Number of agreements"
  ) +
  theme_minimal(base_size = 13) +
  theme(
    plot.title         = element_text(face = "bold", size = 14, margin = margin(b = 4)),
    plot.subtitle      = element_text(color = "grey40", size = 11, margin = margin(b = 12)),
    plot.caption       = element_text(color = "grey60", size = 9, hjust = 0),
    panel.grid.major.x = element_blank(),
    panel.grid.minor   = element_blank(),
    panel.grid.major.y = element_line(color = "grey90"),
    axis.text.x        = element_text(size = 9, color = "grey30"),
    plot.margin        = margin(20, 20, 20, 20)
  )
```




```r
ggsave(file.path(fig_path, "verification_intensity_distribution.png"),
       width = 18, height = 9, dpi = 300)
```

Or, for a bit more detail; create a verbose *weapon_type* and *era* variable:

```
merged_data <- merged_data |>
  mutate(
    weapon_type = case_when(
      weapons_items_topic == 0 ~ "Nuclear",
      weapons_items_topic == 1 ~ "Biological",
      weapons_items_topic == 2 ~ "Chemical",
      weapons_items_topic == 3 ~ "Conventional",
      weapons_items_topic == 4 ~ "Conventional",
      TRUE             ~ NA_character_
    )
  ) |>
  mutate(
    era = case_when(
      year <= 1945 ~ "Pre Cold War",
      year <= 1991 ~ "Cold War",
      year <= 2001 ~ "Post Cold War",
      year > 2001 ~ "Post 9/11"
    )
  )
```

And a color palette to differentiate by weapon type:

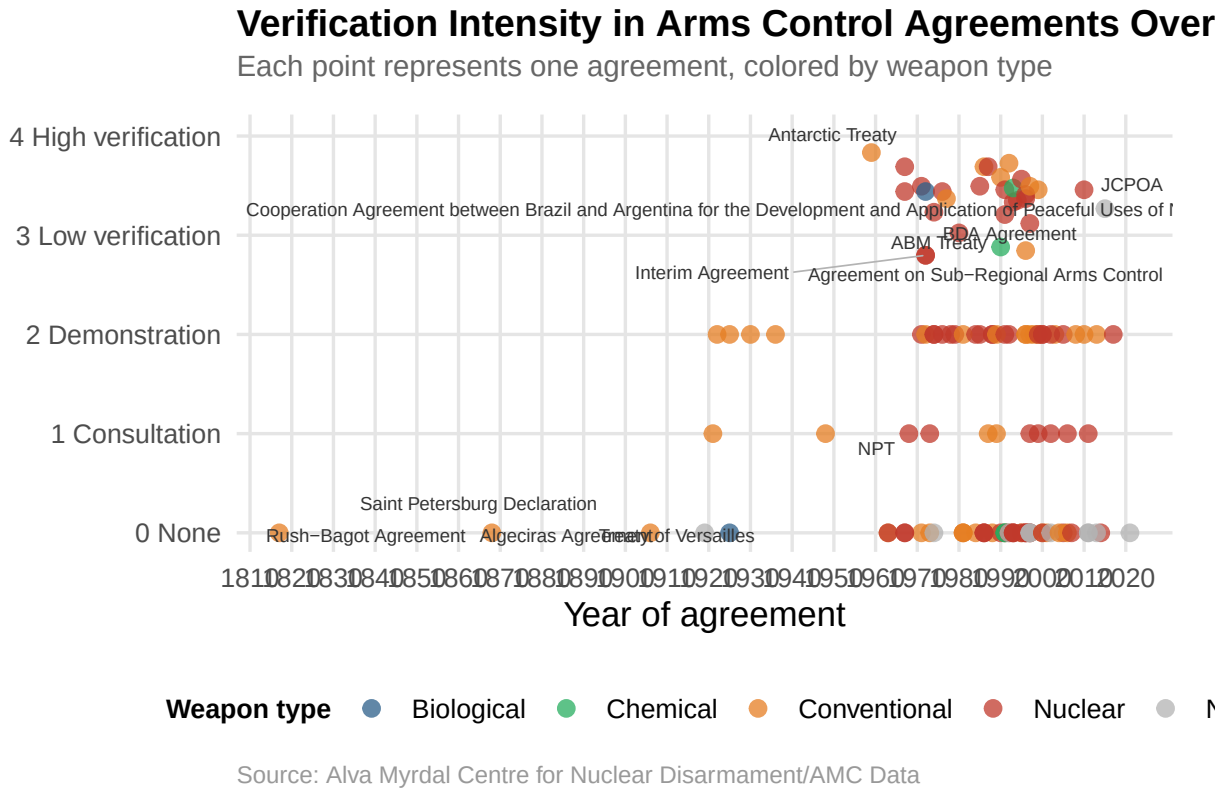
```
weapon_colors <- c(
  "Nuclear"      = "#C1392B",
  "Biological"   = "#2C5F8A",
  "Chemical"     = "#27AE60",
  "Conventional" = "#E67E22"
)
```

And then plot:

```
merged_data |>
  ggplot(aes(x = year, y = extended_verification_intensity, color = weapon_type)) +
  geom_point(size = 2.5, alpha = 0.75) +
  geom_text_repel(
    aes(label = ifelse(
      title_short %in% c("NPT") |   # force NPT to be shown
        extended_verification_intensity >= 2.5 | # label high intensity agreements
        year < 1920,          # label very old agreements
      title_short, ""
    )),
    size          = 2.5,
    max.overlaps  = 15,
    segment.color = "grey70",
    segment.size  = 0.3,
    color         = "grey20"
  ) +
  scale_color_manual(values = weapon_colors, na.value = "grey70") +
  scale_x_continuous(breaks = seq(1810, 2020, by = 10)) +
  scale_y_continuous(
    breaks = 0:4,
```

```r
    labels = c("0 None", "1 Consultation", "2 Demonstration",
               "3 Low verification", "4 High verification"),
    expand = expansion(mult = c(0.05, 0.1))
  ) +
  labs(
    title    = "Verification Intensity in Arms Control Agreements Over Time",
    subtitle = "Each point represents one agreement, colored by weapon type",
    caption  = "Source: Alva Myrdal Centre for Nuclear Disarmament/AMC Data",
    x        = "Year of agreement",
    y        = NULL,
    color    = "Weapon type"
  ) +
  theme_minimal(base_size = 13) +
  theme(
    plot.title       = element_text(face = "bold", size = 14, margin = margin(b = 4)),
    plot.subtitle    = element_text(color = "grey40", size = 11, margin = margin(b = 12)),
    plot.caption     = element_text(color = "grey60", size = 9, hjust = 0),
    panel.grid.minor = element_blank(),
    panel.grid.major = element_line(color = "grey90"),
    legend.position  = "bottom",
    legend.title     = element_text(face = "bold", size = 10),
    axis.text        = element_text(color = "grey30"),
    plot.margin      = margin(16, 16, 16, 16)
  )
```

```
## Warning: Removed 1 row containing missing values or values outside the scale range
## (`geom_text_repel()`).
```



Source: Alva Myrdal Centre for Nuclear Disarmament/AMC Data

```
ggsave(file.path(fig_path, "verification_intensity_over_time.png"),
       width = 18, height = 9, dpi = 300)
```

```
## Warning: Removed 1 row containing missing values or values outside the scale range
## (`geom_text_repel()`).
```

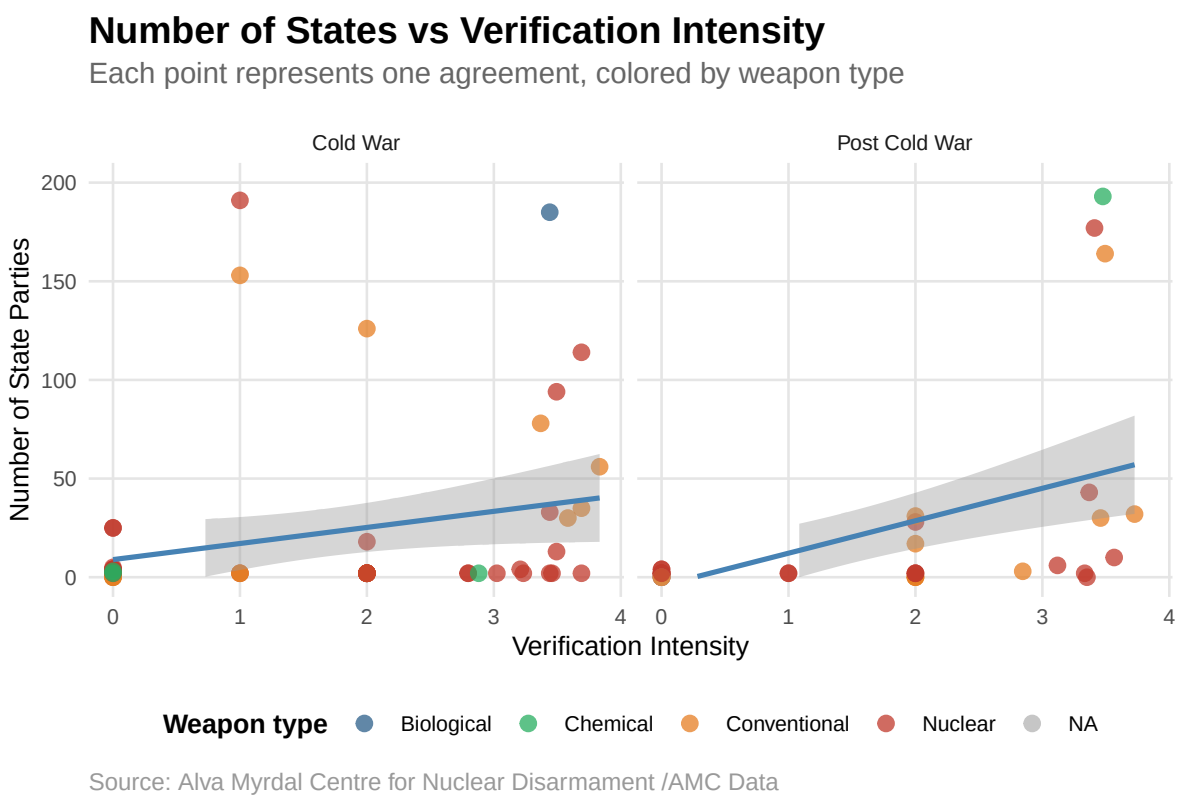
We can then explore the relation between verification intensity and the number of state parties visually:

```
merged_data |>
  #in presentation, only plotted cases with verification intensity over 2
  #filter(extended_verification_intensity > 2) |>
  filter(era == "Post Cold War" | era == "Cold War") |>
    ggplot(aes(
    x = extended_verification_intensity,
    y = nr_states_parties_total,
    color=weapon_type)) +
  geom_point(size = 2.5, alpha = 0.75) +
  scale_color_manual(values = weapon_colors, na.value = "grey70") +
  labs(
    title    = "Number of States vs Verification Intensity",
    subtitle = "Each point represents one agreement, colored by weapon type",
    caption  = "Source: Alva Myrdal Centre for Nuclear Disarmament /AMC Data",
    x        = "Verification Intensity",
    y        = "Number of State Parties",
    color    = "Weapon type"
```

```r
  ) +
  theme_minimal(base_size = 10) +
  facet_wrap(~ era) +
  geom_smooth(method = "lm", se = TRUE, color = "steelblue") +
  ylim(0,200) +
  theme(
    plot.title       = element_text(face = "bold", size = 14, margin = margin(b = 4)),
    plot.subtitle    = element_text(color = "grey40", size = 11, margin = margin(b = 12)),
    plot.caption     = element_text(color = "grey60", size = 9, hjust = 0),
    panel.grid.minor = element_blank(),
    panel.grid.major = element_line(color = "grey90"),
    legend.position  = "bottom",
    legend.title     = element_text(face = "bold", size = 10),
    axis.text        = element_text(color = "grey30"),
    plot.margin      = margin(16, 16, 16, 16)
  )
```

```
## `geom_smooth()` using formula = 'y ~ x'

## Warning: Removed 6 rows containing missing values or values outside the scale range
## (`geom_smooth()`).
```




```r
ggsave(file.path(fig_path, "verification_vs_nstates.png"),
       width = 18, height = 9, dpi = 300)
```

```
## `geom_smooth()` using formula = 'y ~ x'
```

```
## Warning: Removed 6 rows containing missing values or values outside the scale range
## (`geom_smooth()`).
```

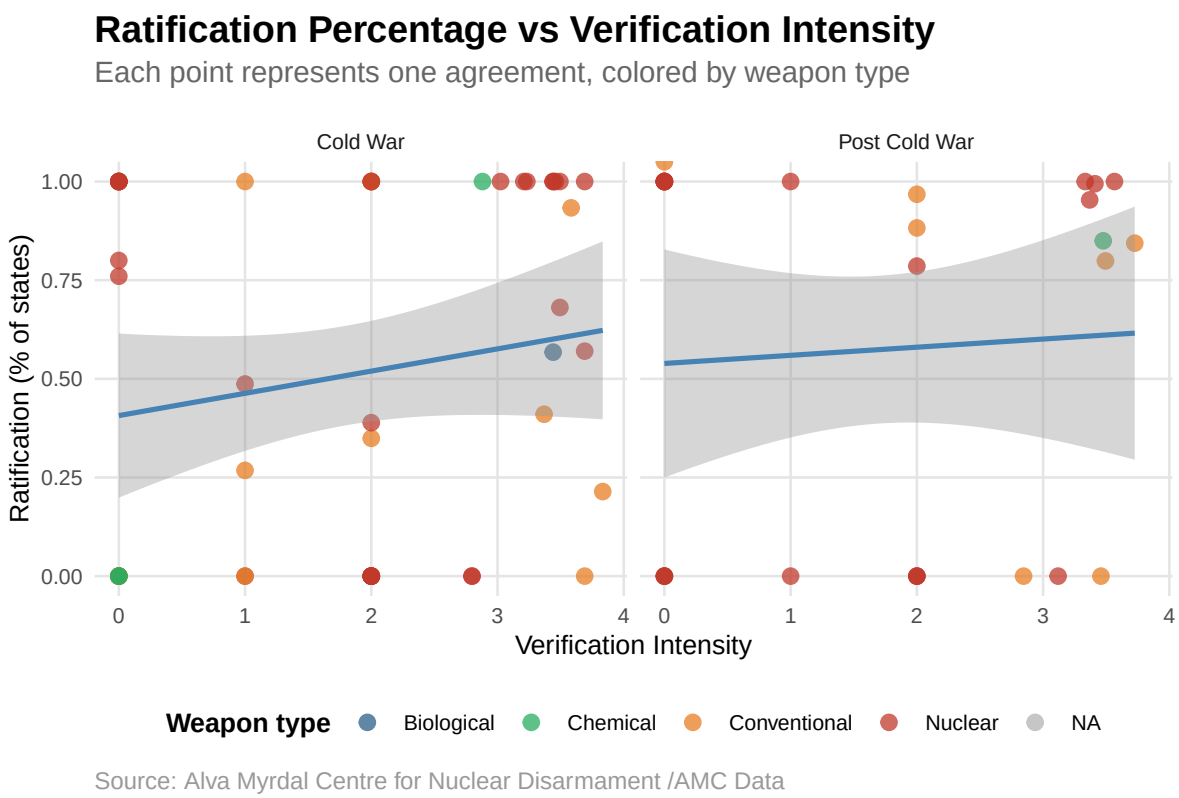
And the same for the ratification percentage:

```r
merged_data |>
  #in presentation, only plotted cases with verification intensity over 2
  #filter(extended_verification_intensity > 2) |>
  filter(era == "Post Cold War" | era == "Cold War") |>
    ggplot(aes(
    x = extended_verification_intensity,
    y = ratification_percentage,
    color=weapon_type)) +
  geom_point(size = 2.5, alpha = 0.75) +
  scale_color_manual(values = weapon_colors, na.value = "grey70") +
  labs(
    title    = "Ratification Percentage vs Verification Intensity",
    subtitle = "Each point represents one agreement, colored by weapon type",
    caption  = "Source: Alva Myrdal Centre for Nuclear Disarmament /AMC Data",
    x        = "Verification Intensity",
    y        = "Ratification (% of states)",
    color    = "Weapon type"
  ) +
  theme_minimal(base_size = 10) +
  facet_wrap(~ era) +
  geom_smooth(method = "lm", se = TRUE, color = "steelblue") +
  ylim(0,1) +
  theme(
    plot.title       = element_text(face = "bold", size = 14, margin = margin(b = 4)),
    plot.subtitle    = element_text(color = "grey40", size = 11, margin = margin(b = 12)),
    plot.caption     = element_text(color = "grey60", size = 9, hjust = 0),
    panel.grid.minor = element_blank(),
    panel.grid.major = element_line(color = "grey90"),
    legend.position  = "bottom",
    legend.title     = element_text(face = "bold", size = 10),
    axis.text        = element_text(color = "grey30"),
    plot.margin      = margin(16, 16, 16, 16)
  )
```

```
## `geom_smooth()` using formula = 'y ~ x'
```

```
## Warning: Removed 16 rows containing non-finite outside the scale range
## (`stat_smooth()`).
```

```
## Warning: Removed 15 rows containing missing values or values outside the scale range
## (`geom_point()`).
```




```
ggsave(file.path(fig_path, "verification_vs_ratification.png"),
       width = 18, height = 9, dpi = 300)
```

```
## `geom_smooth()` using formula = 'y ~ x'

## Warning: Removed 16 rows containing non-finite outside the scale range
## (`stat_smooth()`).
## Removed 15 rows containing missing values or values outside the scale range
## (`geom_point()`).
```

Regression

We can also run a proper linear regression, using the following variables:

Main

- IV = verification intensity
- DV = ratification gap *or* nr_states_parties_total

Controls

- Weapon type
- Era

- Laterality (bi or multi)
- Ban intensity

We thus specify two models, one with ratification percentage as the DV and one with the number of state parties as the DV:

```r
# filter out NAs in DV
model_data <- merged_data |>
  filter(is.finite(ratification_percentage) & !is.na(ratification_percentage)) |>
  mutate(
    weapon_type = as.factor(weapon_type),
    era         = as.factor(era),
    laterality  = as.factor(laterality)
  )

# Ratification model
ratification_model <- lm(
  ratification_percentage ~
    # IV
    extended_verification_intensity +
    # controls
    ban_intensity + weapon_type + era + laterality,
  data = model_data)

# State parties model
n_states_model <- lm(
  nr_states_parties_total ~
    # IV
    extended_verification_intensity +
    # controls
    ban_intensity + weapon_type + era + laterality,
  data = model_data)

# Show model summary
modelsummary(
  list(
    "Number of States" = n_states_model,
    "Ratification Percentage" = ratification_model
  ),
  stars   = TRUE,
  title   = "Verification Intensity and Treaty Participation",
  digits  = 3,
  gof_map = c("nobs", "r.squared", "adj.r.squared")
)
```

Robustness tests

Note: we did not have time to cover these robustness tests in the presentation, but these alternative specifications do suggest that the results found in the main model may be subject to caveats, especially when filtering out agreements that have at least one verification mechanism.

We can run same the regression only for multilateral agreements:

Table 1: Verification Intensity and Treaty Participation

	Number of States	Ratification Percentage
(Intercept)	111.149***	−0.023
	(29.065)	(0.320)
extended_verification_intensity	8.598**	0.009
	(2.911)	(0.032)
ban_intensity	0.754	0.129+
	(6.986)	(0.077)
weapon_typeChemical	−96.635**	0.204
	(33.858)	(0.373)
weapon_typeConventional	−115.094***	0.183
	(27.804)	(0.306)
weapon_typeNuclear	−128.487***	0.388
	(28.568)	(0.314)
eraPost 9/11	2.854	0.157
	(11.932)	(0.131)
eraPost Cold War	5.283	0.027
	(8.994)	(0.099)
eraPre Cold War	−34.372*	0.100
	(16.175)	(0.178)
laterality2	40.204***	0.385***
	(8.287)	(0.091)
Num.Obs.	100	100
R2	0.441	0.256
R2 Adj.	0.386	0.182

+ p <0.1, * p <0.05, ** p <0.01, *** p <0.001

```r
model_data <- merged_data |>
  filter(is.finite(ratification_percentage) & !is.na(ratification_percentage)) |>
  filter(laterality == 2) |>
  mutate(
    weapon_type = as.factor(weapon_type),
    era         = as.factor(era),
  )

lat_ratification_model <- lm(
  ratification_percentage ~
    # IV
    extended_verification_intensity +
    # controls
    ban_intensity + weapon_type + era,
  data = model_data)

lat_n_states_model <- lm(
  nr_states_parties_total ~
    # IV
    extended_verification_intensity +
    # controls
    ban_intensity + weapon_type + era,
  data = model_data)

modelsummary(
  list(
    "Number of states" = lat_n_states_model,
    "Ratification Percentage" = lat_ratification_model
  ),
  stars   = TRUE,
  title   = "Verification Intensity and Treaty Participation (multilateral only)",
  digits  = 3,
  gof_map = c("nobs", "r.squared", "adj.r.squared")
)
```

Or only for bilateral agreements:

```r
model_data <- merged_data |>
  filter(is.finite(ratification_percentage) & !is.na(ratification_percentage)) |>
  filter(laterality == 1) |>
  mutate(
    weapon_type = as.factor(weapon_type),
    era         = as.factor(era),
  )

bilat_ratification_model <- lm(
  ratification_percentage ~
    # IV
    extended_verification_intensity +
    # controls
    ban_intensity + weapon_type + era,
  data = model_data)
```

Table 2: Verification Intensity and Treaty Participation (multilateral only)

	Number of states	Ratification Percentage
(Intercept)	142.167**	0.410
	(40.880)	(0.244)
extended_verification_intensity	13.373*	−0.035
	(5.599)	(0.033)
ban_intensity	6.343	0.050
	(16.587)	(0.099)
weapon_typeChemical	−70.143	−0.062
	(55.838)	(0.333)
weapon_typeConventional	−113.756**	0.134
	(38.648)	(0.231)
weapon_typeNuclear	−126.926**	0.425+
	(40.787)	(0.243)
eraPost 9/11	0.451	0.303*
	(23.571)	(0.141)
eraPost Cold War	−7.221	0.174
	(18.299)	(0.109)
eraPre Cold War	−33.048	0.163
	(25.959)	(0.155)
Num.Obs.	53	53
R2	0.337	0.340
R2 Adj.	0.216	0.220

+ p <0.1, * p <0.05, ** p <0.01, *** p <0.001

Table 3: Verification Intensity and Treaty Participation (bilateral only)

	Number of states	Ratification Percentage
(Intercept)	1.749***	0.299
	(0.427)	(0.360)
extended_verification_intensity	0.172+	0.135+
	(0.089)	(0.075)
ban_intensity	0.047	0.121
	(0.141)	(0.119)
weapon_typeConventional	0.136	−0.167
	(0.456)	(0.385)
weapon_typeNuclear	−0.184	−0.230
	(0.439)	(0.370)
eraPost 9/11	0.173	0.116
	(0.303)	(0.256)
eraPost Cold War	0.545*	0.149
	(0.238)	(0.201)
Num.Obs.	47	47
R2	0.152	0.148
R2 Adj.	0.024	0.020

+ p <0.1, * p <0.05, ** p <0.01, *** p <0.001

```
bilat_n_states_model <- lm(
  nr_states_parties_total ~
    # IV
    extended_verification_intensity +
    # controls
    ban_intensity + weapon_type + era,
  data = model_data)

modelsummary(
  list(
    "Number of states" = bilat_n_states_model,
    "Ratification Percentage" = bilat_ratification_model
  ),
  stars   = TRUE,
  title   = "Verification Intensity and Treaty Participation (bilateral only)",
  digits  = 3,
  gof_map = c("nobs", "r.squared", "adj.r.squared")
)
```

Or only for the Post-WW2 eras:

```
model_data <- merged_data |>
  filter(is.finite(ratification_percentage) & !is.na(ratification_percentage)) |>
  filter(era != "Pre Cold War") |>
  mutate(
```

```r
    weapon_type = as.factor(weapon_type),
    era         = as.factor(era),
    laterality  = as.factor(laterality)
  )

filt_ratification_model <- lm(
  ratification_percentage ~
    # IV
    extended_verification_intensity +
    # controls
    ban_intensity + weapon_type + era + laterality,
  data = model_data)

filt_n_states_model <- lm(
  nr_states_parties_total ~
    # IV
    extended_verification_intensity +
    # controls
    ban_intensity + weapon_type + era + laterality,
  data = model_data)

modelsummary(
  list(
    "Number of States" = filt_n_states_model,
    "Ratification Percentage" = filt_ratification_model
  ),
  stars   = TRUE,
  title   = "Verification Intensity and Treaty Participation (Post WWII only)",
  digits  = 3,
  gof_map = c("nobs", "r.squared", "adj.r.squared")
)
```

Or only for agreements with a verification intensity level of at least 2:

```r
model_data <- merged_data |>
  filter(is.finite(ratification_percentage) & !is.na(ratification_percentage)) |>
  filter(n_verification_mechanisms > 0) |>
  mutate(
    weapon_type = as.factor(weapon_type),
    era         = as.factor(era),
    laterality  = as.factor(laterality)
  )

filt_ratification_model <- lm(
  ratification_percentage ~
    # IV
    extended_verification_intensity +
    # controls
    ban_intensity + weapon_type + era + laterality,
  data = model_data)

filt_n_states_model <- lm(
  nr_states_parties_total ~
```

Table 4: Verification Intensity and Treaty Participation (Post WWII only)

	Number of States	Ratification Percentage
(Intercept)	114.235**	0.137
	(41.071)	(0.433)
extended_verification_intensity	8.876**	0.014
	(3.127)	(0.033)
ban_intensity	−0.005	0.120
	(7.664)	(0.081)
weapon_typeChemical	−99.795*	0.041
	(44.417)	(0.469)
weapon_typeConventional	−118.851**	0.007
	(40.157)	(0.424)
weapon_typeNuclear	−131.840**	0.221
	(40.254)	(0.425)
eraPost 9/11	3.157	0.165
	(12.363)	(0.130)
eraPost Cold War	5.369	0.030
	(9.318)	(0.098)
laterality2	40.226***	0.382***
	(8.596)	(0.091)
Num.Obs.	92	92
R2	0.424	0.267
R2 Adj.	0.368	0.197

+ p <0.1, * p <0.05, ** p <0.01, *** p <0.001

Table 5: Verification Intensity and Treaty Participation (At least 1 verification mechansism)

	Number of States	Ratification Percentage
(Intercept)	−18.010	−1.583
	(136.474)	(1.046)
extended_verification_intensity	43.213	0.619+
	(40.191)	(0.308)
ban_intensity	−2.019	0.032
	(11.924)	(0.091)
weapon_typeChemical	−59.512	0.515
	(61.753)	(0.473)
weapon_typeConventional	−145.131*	−0.211
	(52.347)	(0.401)
weapon_typeNuclear	−126.664*	0.267
	(52.196)	(0.400)
eraPost 9/11	−2.546	0.172
	(52.936)	(0.406)
eraPost Cold War	23.016	−0.002
	(20.488)	(0.157)
laterality2	54.338+	0.021
	(26.135)	(0.200)
Num.Obs.	29	29
R2	0.568	0.407
R2 Adj.	0.395	0.170

+ p <0.1, * p <0.05, ** p <0.01, *** p <0.001

```
    # IV
    extended_verification_intensity +
    # controls
    ban_intensity + weapon_type + era + laterality,
  data = model_data)

modelsummary(
  list(
    "Number of States" = filt_n_states_model,
    "Ratification Percentage" = filt_ratification_model
  ),
  stars   = TRUE,
  title   = "Verification Intensity and Treaty Participation (At least 1 verification mechansism)",
  digits  = 3,
  gof_map = c("nobs", "r.squared", "adj.r.squared")
)
```